

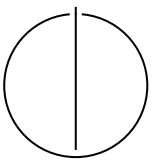
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

A Unikernel-based OS for On-Device LLM Inference

Simon Kammermeier





SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

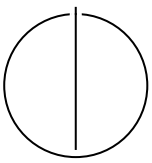
TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

A Unikernel-based OS for On-Device LLM Inference

Ein Unikernel-basiertes Betriebssystem für On-Device-LLM-Inferenz

Author:	Simon Kammermeier
Examiner:	Prof. Pramod Bhatotia
Supervisor:	Teofil Bodea, Ilya Meignan--Masson
Submission Date:	12. 02. 2026



I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, 12. 02. 2026 Simon Kammermeier

Acknowledgments

First, I would like to thank my advisors, Teofil Bodea and Ilya Meignan--Masson for their invaluable help throughout this entire process. Their expertise on the topic and support in execution were invaluable in pushing this work forward. This thesis wouldn't have been possible without their encouragement.

Second, I would like to thank Prof. Bhatotia for giving me the possibility to work on this topic. His close mentorship and constant push for the best results were central to the quality of my work.

Last, I would like to thank my parents for always being there. They allow me to pursue my dreams and offer constant support, company, and much-needed distractions.

Abstract

On-device large language model (LLM) inference executes machine learning tasks directly on the user's hardware, rather than relying on remote cloud infrastructure. This approach offers advantages in preserving data privacy, reducing response latency, and robustness against network unavailability. Despite recent efforts to reduce resource requirements, the constraints inherent to mobile devices limit achievable performance and efficiency.

We present LROS, an end-to-end system for on-device LLM inference. LROS leverages virtualization to provide a standalone service for LLM inference at the operating system level. The unikernel architecture enables close integration between the inference engine and the specialized libOS. A custom memory-mapping mechanism optimizes model weight access in memory-constrained scenarios by applying customizable policies to control the order of file prefetch. This provides the unique ability to adapt to the model's architecture. The integration of vAccel enables simple, efficient offloading of matrix calculations to compute accelerators. Its modular design and generic interface allow for easy adaptation to diverse compute platforms with minimal code changes.

We demonstrate the adaptability of LROS by testing on a GPU-based edge platform and a mobile device including an NPU. The custom memory prefetcher achieves up to 32.8$\times$ higher inference throughput than a baseline Linux virtual machine.

Contents

Acknowledgments **iii**

Abstract **iv**

1 Introduction **1**
1.1 A Case for On-device Inference 1
1.2 State-of-the-Art and Research Gap 2
1.3 Our Approach 3

2 Background **6**
2.1 Large Language Models 6
2.2 On-device LLM Inference 8
2.3 Accelerator 9

3 Overview **11**
3.1 System Overview 11
3.2 Life of a Request 12
3.3 Programming Model 13
3.4 Key Design Challenges 13

4 Design **16**
4.1 LLM Orchestration: Model and LoRA Selection 16
4.2 Lightweight Library OS 17
4.3 Memory Management 19
4.4 Accelerator Integration: vAccel 20

5 Implementation **23**
5.1 Unikraft 23
5.2 Inference Engine: llama.cpp 25
5.3 vAccel 25
5.4 vAccel Operations 28

6 Evaluation **31**
6.1 Experiment Setup 31

6.2 Light-weight OS 31
6.3 OS Specializations 33
6.4 Accelerator Integration: vAccel 35

7 Related Work **37**

8 Conclusion **39**

List of Figures **40**

List of Tables **41**

Bibliography **42**

1 Introduction

In recent years, various industries have widely adopted machine learning, particularly large language models (LLMs). Chatbots, such as ChatGPT by OpenAI or Google Gemini, offer user-friendly access to this technology. The ease of use and ever-growing capabilities in summarizing, rephrasing natural language text, answering questions, and generating source code have attracted both mainstream and professional audiences. The high quality of the generated text has been reached in various ways. First, the model sizes were massively scaled up. In addition, the training data set has been expanded. Both of these actions increase the required computing power. Only the vast computing capabilities available via cloud computing have enabled this trend.

1.1 A Case for On-device Inference

Mobile-friendly interfaces helped drive seamless integration of LLM services into various workflows. In fact, smartphones can greatly benefit from LLM capabilities. Summarizing long articles or assisting with drafting message replies are LLM capabilities relevant to some of the most common smartphone activities. As such, it is a natural conclusion to try using mobile devices directly for the LLM inference. In contrast to cloud services, shifting the required computations to the user offers several advantages. First, no user data needs to be sent to external services, which offers a privacy advantage. Secondly, eliminating server access also improves the user's quality of experience by providing more stable performance and greater robustness in the event of connection failures.

Challenges of on-device inference. While on-device inference is a promising approach [Yua+24; Li+24b; Yin+25], moving beyond the elastic, seamless scalability of the cloud remains a major challenge for LLM inference. Mobile devices are resource-limited in terms of available computation power, installed system memory, and power budget. Several techniques have been introduced that address resource limitations at the model level [HMD16; Jac+18; Fra+23]. Quantization [Jac+18] trades precision on the model weights for lower memory usage. Still, there are limits to this technique, which necessitate augmenting it with advanced strategies to load weights on demand [Ali+24]. The operating system can supply interfaces that transparently handle the required IO

operations.

Additionally, on-device inference does not run exclusively. It is colocated with many other processes that enable central functions requested by the user. However, addressing this resource competition requires dynamic knowledge only available at the operating system level. Any complete system thus needs to consider improvements both at the model and operating system levels.

1.2 State-of-the-Art and Research Gap

LLMs have proven capable of enabling serious innovations, but their implementation in on-device settings faces significant challenges. Existing solutions do not account for the specific requirements of on-device inference under resource constraints. [Li+24a; Goo26; Gun+24; App26] Typically, existing inference solutions only consider scenarios where they run exclusively on the device. In that case, unique device ownership allows limitations to be naturally enforced by hardware constraints. This model does not accurately reflect reality. The high resource demands of LLM inference [Xu+23] will monopolize system resources, negatively impacting user experience by delaying other tasks running on the device.

At the application level, it is hard to effectively limit resource usage. To influence the amount of CPU time used, the application can choose different numbers of worker threads. Often, background tasks, such as request retrieval or logging, are handled by additional threads. This will complicate tracking the amount of compute time used. On the other hand, memory usage can be influenced by manually accounting for all compute buffers. The model weights then need to be handled using a user-space cache, which allows using a limited buffer size. This imposes significant work on the developer of the inference engine. Additionally, many programming languages' basic constructs and data structures rely on heap allocation. In general, it is not easily possible to keep track of this. Especially when using standard library data structures, the associated memory overhead can be substantial.

Instead, tracking resource usage at the operating system level allows the inference engine to run without code changes. Additionally, the operating system can more accurately assess the resource requirements of individual processes, since ultimately all assignments of hardware resources need to be made via operating system interfaces. In this way, fragmented allocations and background threads can be handled naturally. On top of that, additional metrics, such as power usage and thermals, which lack standard user-space interfaces, can be read directly from hardware sensors and incorporated into scheduling decisions.

Important industry actors are increasingly moving towards on-device LLMs. Google

has integrated `AI Core` [Goo26] into Android. It provides a system service that runs LLM inference, among other AI tasks, on behalf of apps that use the `ML Kit GenAI` API. This service only supports a limited set of devices. Similarly, `Apple Intelligence` [Gun+24] also offers on-device ML. Third-party developers can access it via the `Foundation Models framework`. Due to the proprietary nature of both of these solutions, only a specific model chosen by the respective vendor is available for inference. As an alternative deployment model, Apple also provides the `Core ML` API [App26], which allows developers to bring their own models. This older API was not designed with the size of current LLMs in mind. Every app packages its own model, which must be loaded into memory and consumes disk space. The limited system memory only allows very small models to run successfully with this interface.

The heterogeneity of mobile platforms is one of the root problems complicating the local deployment of LLMs. In particular, differences in the computing capabilities of installed hardware make it difficult to supply a generally efficient implementation. On the CPU side, the Arm architecture provides a common foundation for all implementations.

Since LLM inference relies on accelerators for speed and efficiency, it is important to exploit them as well. The underlying hardware of GPUs can differ significantly in both performance and, possibly, the actual programming model. There are standard APIs for GPUs, such as OpenCL, that provide a common programming interface. In contrast, this cannot be applied to specialized AI accelerators like TPUs or NPUs, which are highly vendor-specific. Their various offerings in supported computations lack established standards. As a result, implementing a general code path is impossible. It is necessary to adapt it for every hardware combination.

The root cause of this problem is in the deployment model. The tight integration into the operating system makes it impossible to tune the implementation to the hardware without modifying the core operating system. In particular, hardware vendors can only provide support for their specialized NPUs by adapting the inference implementation. This is only possible by changing the system service and, therefore, the operating system. Since the device and software vendor are different, this prohibits simple adaptation.

To this end, we ask the following research question: *How can we design a system to enable performant and efficient LLM inference in an on-device environment, while respecting resource constraints inherent to this setting?*

1.3 Our Approach

The partitioning of hardware resources among processes with differing requirements is a challenging problem, as different workloads place uneven, time-varying demands on

CPUs, memory, caches, and I/O devices. Any static or fine-grained partitioning risks either underutilizing resources or allowing interference between tasks. Virtualization is a natural fit for this problem, as it enables the partitioning of resources between multiple distinct sets of processes. This allows us to impose resource limits on applications and enables isolation.

To address this problem, we propose LROS, an end-to-end inference service. We change the deployment model to run the system using *virtualization*. The resulting isolation enables the enforcement of resource limitations. This prevents LLM inference from monopolizing the device and degrading the user experience of concurrent tasks.

LROS is based on the *unikernel* architecture. Composing the inference engine and a light-weight operating system into a tightly integrated image reduces performance overheads and enables specializations of system functions according to application needs.

The custom *memory manager* enables efficient loading of model weights into memory (Section 4.3). It includes an intelligent prefetching mechanism that makes weights available when needed. In this way, it maximizes inference performance in the memory-constrained environment inherent to mobile settings.

LROS integrates *vAccel* to allow flexible access to accelerators, such as NPUs or GPUs (Section 4.4). This modular framework enables XPU access in our virtualized environment. Its device-agnostic API is backed by hardware-specific implementations in dynamically loaded plugins.

The *orchestrator* supports using different models and LoRA per request (Section 4.1). It coordinates all the steps necessary to provide the system service. This includes selecting the model and LoRA. It also handles the scheduling of inference actions. Lastly, it communicates with the host operating system to receive new requests and return results.

Evaluation. We evaluate LROS based on two platforms. We evaluate for mobile devices with an NPU and more powerful edge devices equipped with a GPU. Our design achieves significant improvements over a baseline Linux VM. The custom memory prefetcher offers 32.8× higher throughput than the baseline in memory-limited scenarios and decreases load times by up to 93%.

Contributions. Overall, our contributions are as follows:

- We propose a light-weight unikernel image to provide an LLM inference system service.
- We design and implement a memory manager that efficiently loads the inference model and manages its allocation. Its behavior is customizable by model-dependent policies.

- We provide a generic, flexible access mechanism for accelerators in virtualized environments. Using a modular design and device-agnostic API, we support the heterogeneous XPU landscape.

The project's source code is available at `https://www.github.com/TUM-DSE/lros`.

2 Background

2.1 Large Language Models

Large language models (LLMs) are machine learning systems trained on vast corpora of text. They learn the patterns of natural language to process and generate texts that appear human and represent coherent thought.

LLMs use their training weights, the prompt, and the previously generated tokens to predict a suitable next token.

LLMs use huge amounts of data in the form of their so-called model weights. These billions of parameters absorb information from the training data and store the LLM's knowledge. The data type used for the parameters affects the model's capabilities. Larger ranges for the data type improve calculation accuracy, yielding more accurate answers. On the other hand, the increased size has higher demands on the hardware and increases the computational load.

The weights are organized into tensors. Mathematically, these are a generalization of vectors and matrices to an arbitrary number of dimensions. Essentially, all calculations of LLMs correspond to operations involving one or more of these vectors. The most common operations are matrix multiplications and vector additions.

The lifetime of an LLM can be split into two main stages: training and inference. During the training, the weights are gradually adjusted to enable the language model to accurately reproduce the training dataset. Due to the size of the model and, more importantly, the amount of training data, this process requires immense computational resources and is therefore very expensive. The inference uses the weights obtained during training. It takes an input prompt, processes it, and then generates new tokens that best fit the model. In this stage, the model weights are no longer updated, and all requests use the same values.

In both stages, the same computations take place. Almost all modern LLM architectures are based on the Transformer [Vas+17] architecture. The input text is converted into tokens, where each token is a number that corresponds to a word or to semantically relevant parts of a word, such as a prefix or suffix. Using the embedding tensor, essentially a lookup table of embeddings for each possible token, they are converted to high-dimensional vectors before adding the positional encoding, which encodes the token's position within the input sequence.

The meat of a transformer model is in the encoder and decoder layers. Most relevant text generation models are decoder-only and lack encoder layers. Each layer has the same structure. First, the so-called self-attention sublayer captures a token's relations with those that precede it in the input sequence. Linear transformations take the embedding vectors and generate three vectors for each token: Key (K), Value (V), and Query (Q). Then, dot products are calculated between the Q vector of the current token and the K vector of all previous tokens. The resulting scalars are used as attention weights to calculate a weighted average of the V vectors, which is then added to the embedding vector. Next, this vector is further processed in the feed-forward network. Here, a linear transformation expands the vector into a higher-dimensional intermediate space. The activation function is applied individually to each value in the vector. Then another linear transformation projects back down to the original dimension. At this point, the output is fed to the attention mechanism of the next layer.

After all layers have been applied, the resulting vector needs to be converted back into a token corresponding to (part of) a word. To reverse the embedding, the vector resulting from applying all previous transformations is expanded into a probability distribution over the language model's vocabulary. A linear transformation maps the vector from the embedding space to the significantly larger vocabulary space. Then, applying the softmax function converts this to a probability vector. Lastly, the next token is sampled from this distribution.

This description glances over some details, such as the application of normalizations and slight differences between model architectures. The basic knowledge presented here is sufficient to understand this project.

One important insight here is that the K and V vectors in the attention calculations depend only on the token to which they belong. So they are unchanged between multiple iterations of the model. To avoid recalculating them, the KV cache [Pop+22] stores these values after they are initially computed.

The process of LLM inference consists of two distinct phases. The prompt is the input of each inference query. Since it is fully known at the beginning of the calculation, there are no dependencies between the computations, and they can be done in parallel. During this prefill phase, the KV cache and other internal data structures are populated, and the first output token is generated. The decode phase generates tokens one at a time. Each full iteration of the model generates one new token, which is then concatenated to the existing input sequence, before the next iteration generates the next token. This auto-regressive process is largely sequential and, therefore, less computationally intensive.

2.2 On-device LLM Inference

The software component that executes LLM queries on a trained model is called the inference engine. The resource-intensive training process of an LLM can only be completed in a reasonable time on cloud server farms, which allow the use of hundreds or even thousands of GPUs in parallel to gradually adjust model weights. For example, Meta used 16k H100 GPUs for training the Llama 3 family of models [Gra+24]. The training process consumes hundreds of thousands of GPU hours.

While the inference has much lower computational requirements, the large data size still imposes a significant hardware load and constraints on memory usage. Accordingly, the traditional deployment strategy for LLMs involves introducing a web service that orchestrates user interactions with cloud computing instances that provide the actual LLM inference service.

However, the involvement of an external server raises concerns regarding data privacy. The queries and answers given by the LLM are available in plain text at the inference engine. The increasing use of AI chatbots for sensitive applications, such as assisting with medical questions or providing support in financial decisions, further heightens these concerns. Additionally, messages exchanged between chatbots and users are often recorded for future model training to enhance the LLM's performance. Any contained private data may be inadvertently leaked in future conversations.

Moreover, relying on cloud services makes it hard to reliably predict performance. Queues on the server backends, which are necessary to smooth out server load, can cause delays in request processing. Another potential source of problems can be packet loss or processing delays on the network connection. Expanding on this, cloud inference can only be applied when the user has a stable internet connection.

As an alternative, transferring inference tasks from the cloud closer to the user can mitigate these problems. Processing on the end-user's device gives full control over the data. It eliminates network transfer, allowing it to be used even without an internet connection. Consequently, it removes the need for companies to purchase server capacity to host the inference service.

In contrast to cloud services, on-device inference faces different challenges. In particular, resources are constrained on several dimensions. One concern is power usage. Mobile devices have limited energy budgets that must be preserved to achieve acceptable battery life and charging intervals for the user. Another dimension of this restriction is thermals. Mobile devices typically use passive cooling, which severely limits their heat dissipation capacity. Due to their handheld nature, they also have tight thermal targets that have to be respected.

The installed memory on mobile devices is very low in comparison to the size of LLMs. Recent advances in LLM architecture and inference algorithms offer solutions to

this. In particular, they enabled a significant reduction in model memory size while preserving their abilities.

Quantization [Jac+18] reduces the size of each weight parameter. Often, the dynamic range of the weights is relatively low, allowing them to be represented using smaller data types without significant loss of accuracy. More advanced quantization schemes have further improved this approach. Applying different offsets and scaling to different parts of the weights increases flexibility. Knowledge distillation [HVD15] and weight pruning [Han+15] enable the reduction of the number of weights.

User devices are typically not fully dedicated to LLM inference. Instead, they are shared by many diverse tasks. This multi-tenancy means the inference cannot monopolize system resources. For an optimal user experience, it is necessary to balance resource usage so that inference performance remains acceptable without compromising system performance.

2.3 Accelerator

LLM inference is a computationally heavy task that involves highly parallel computation. Such computations have characteristics different from those of general-purpose calculations. They require massively parallel execution of the same instruction and rarely require branches.

Graphical processing units (GPUs), while initially designed to evaluate shader calculations for rendering virtual scenes and producing graphics, perform these types of calculations. In recent times, processors designed specifically for AI calculations have emerged, such as neural processing units (NPUs) or tensor processing units (TPUs). In comparison to GPUs, these are simpler, omitting rendering-related functionality, and are tuned for energy efficiency. Therefore, they are particularly relevant for low-cost devices or use cases with high energy-efficiency requirements. Consequently, the choice of the right accelerator depends heavily on your specific requirements. Mobile devices may want a low-power, low-cost NPU, while an edge device that can afford a higher power budget and higher costs may want to use a GPU.

Complications arise when combining virtualization with the use of accelerator devices. Making the processing available for use within the VM can be done in multiple ways.

Full emulation is generally infeasible due to high performance overheads and the unavailability of information on the necessary hardware details. A second way would be to pass the device through to the host. This technique is widely used, but it has significant limitations. Only high-end server-grade GPUs allow pass-through to multiple VMs; otherwise, the device cannot be shared. In addition, only platforms with an IOMMU can exploit this, which excludes many mobile and edge platforms,

such as the NVIDIA Jetson AGX Orin and the OrangePi 5 Ultra. Another complication is that pass-through requires a driver inside the VMs, which involves a significant development effort and results in a substantial increase in image and code size.

Another possibility to enable access is to perform API remoting. API calls in the VM are forwarded to the host via a remote function call. The host then executes the corresponding action on behalf of the guest. This alleviates several of the previously described issues. It allows sharing the device with multiple VMs and the host, just as multiple applications share the device when running concurrently. The guest OS does not require its own device driver. Instead, it just needs an implementation of the API forwarding protocol. No special hardware (like an IOMMU) is needed. As a drawback, typical APIs require many fine-grained entry points, which increases development effort and the overhead of remote calls.

3 Overview

3.1 System Overview

We present LROS, an end-to-end solution for on-device LLM inference. LROS provides a service for apps running on the device, enabling them to submit LLM inference queries and have them completed efficiently. LROS is deployed as a virtual machine to enjoy performance isolation from the rest of the system. For this purpose, LROS uses a customized OS with an LLM-optimized memory manager. LROS ensures access to specialized accelerators via vAccel, a novel, modular framework that exposes hardware-acceleration functionality to virtualized environments.

Virtualization. LROS uses virtualization to create a software system that is fully independent of the host. In mobile settings, user experience is highly dependent on system responsiveness. As such, it is important to balance resource usage across concurrent tasks on the device. LLM inference cannot effectively do this at the application level. Instead, running the system in a VM enables device vendors to easily impose resource limits, since all used resources must be explicitly assigned by the hypervisor. A customized operating system incorporates modifications and optimizations to enable efficient, high-performance LLM inference.

Inference engine. The inference engine provides the user-facing functionality. It uses the operating system libraries to execute inference requests received from applications running outside LROS.

Accelerator access & management: vAccel. vAccel enables the system to access compute accelerators. It provides a virtualized API that exposes generic operations for offloading calculations to external devices. This serves two main purposes in our design. First, the generic interface enables transparent adaptation to various device types without requiring changes to the inference engine's implementation. This enables simple adaptation to heterogeneous target platforms, including those with GPU- or NPU-based acceleration. Second, the encapsulation of LROS in a virtualized environment requires special protocols to enable the usage of compute devices.

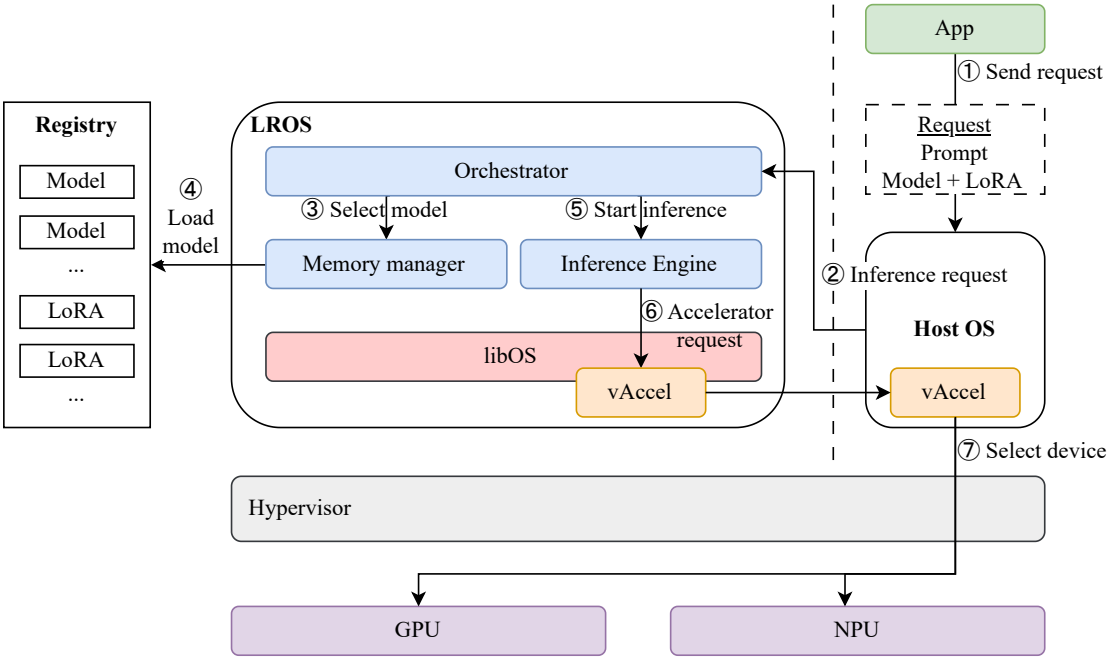


Figure 3.1: LROS overview. LROS *composes the Inference Engine and the specialized libOS into a lightweight unikernel.*

3.2 Life of a Request

System Workflow. The inference workflow using LROS, shown in Figure 3.1, therefore, consists of multiple steps: The user application running outside of LROS on the host OS submits an inference request (①). It consists of a prompt string and can optionally specify a token count to limit the answer's length. To obtain better-tuned results, the query can optionally specify the identifiers of the requested model and LoRA adapters to use for inference. In step ②, the host operating system forwards the request to the LROS orchestrator component. The orchestrator parses the request, then selects the required model and LoRA adapters (③). If the currently active model differs from the requested one, the memory manager loads the model from storage (④). Next in step ⑤, the orchestrator starts the inference using the included inference engine. Some of the calculations involved are suitable for offloading to accelerators. The engine uses the vAccel interface to request accelerator computation (⑥).

The driver in the libOS transmits them to the host operating system. There, the vAccel runtime component unpackages the request. It selects an applicable vAccel backend implementation for the operation (⑦). This choice determines the device

Model Registry	
registerModel(*model*) → *m_id*	Register model for inference
registerLoRA(*LoRA*, *m_id*) → *l_id*	Register LoRA for model
Inference	
createReq(*prompt*, *maxTokens*) → *req*	Create a request for inference
selectModel(*req*, *m_id*)	Select specific model for request
addLora(*req*, *l_id*, *weight*)	Add LoRA to request at given weight
infer(*req*) → *res*	Start inference request

Table 3.1: LROS system API

that executes the computation. The execution status and other potential outputs are returned to the runtime, which transmits them back to the vAccel driver in the LROS kernel. Then, they are relayed to the inference engine to complete the request.

3.3 Programming Model

LROS provides inference as a system service. As such, its programming model is tailored to this use case.

The provided system APIs (Table 3.1) expose the inference service to the user. The *model registry* APIs allow registering models and associated LoRAs to make them available for subsequent requests. They are identified using the model and LoRA IDs. The *inference* APIs provide access to the actual inference service. The user can create a request for a specified prompt, optionally limiting the token count. For every request, the user may additionally override the model to a previously registered one. They can also add registered LoRAs at a chosen weight. To receive results, the request is submitted to the system.

3.4 Key Design Challenges

With our design, we aim to address the following four design challenges.

Challenge #1: Lightweight OS design. Traditional operating systems provide capabilities applicable in a wide variety of use cases. As a result, they introduce many subsystems that provide functionality inconsequential to the specific application of LLM inference. Therefore, their footprint is substantial and significantly larger than necessary when considering only a single use case. Other measures of efficiency also

suffer from the generality. As such, boot times are slow, and overhead introduced by abstractions may limit achievable throughput or latency.

The Linux kernel provides an extensive set of configuration options at compile time. This allows fine-tuning behavior and excluding unwanted functionality. While the number of tunable parameters is quite vast, many components are required to provide a suitable execution environment. As such, the actual extent of available tunability is limited. Additionally, the base architecture provides limitations that cannot be overcome just by excluding subsystems.

We take a different approach and use a different base architecture. Employing a modular unikernel enables us to eliminate unnecessary functionality. This architectural design eliminates the central limitations of traditional operating systems, such as the address space split between kernel and user space.

Challenge #2: Suboptimal OS interfaces. The interfaces and functionality that general-purpose operating systems provide to applications are designed to work well in most cases. It is not possible to properly represent application-specific behavior using standard interfaces, such as those provided by POSIX.

For example, the `mmap` system call provides a way to create memory mappings, which include mappings of a file to a memory region. In this way, a process can use memory access to read from a file, and the operating system transparently handles the required I/O operations. This has the advantage that the kernel does not need to copy data from the kernel to user buffers, as is required for `read` calls. In the event of memory pressure, file-backed pages can be easily evicted and re-fetched if needed. Only the parts of files that are actually accessed occupy memory. While this interface provides an automatic way to cache file accesses without any user-side implementation, it does not adapt to complex access patterns. It is possible to use `madvise` to communicate simple access patterns to the kernel from user space. The kernel, however, may just ignore the hint. Only generic policies are allowed.

Our design follows the unikernel approach, enabling us to implement custom interfaces without the limitations imposed by address-space isolation and the system call ABI. Callback functions from the kernel to the user space can express custom policies.

Challenge #3: Access to accelerators in a virtualized environment. LROS aims to enable efficient, high-performance inference. However, CPU-based execution is not sufficient to achieve this goal. In contrast, the isolation provided by virtual machines, while meeting the requirement of resource limitation, significantly complicates access to accelerator devices. The heterogeneity of accelerators and their non-standard APIs introduces additional difficulties.

There are traditional mechanisms for accessing accelerator devices in a VM. Pass-through exclusively assigns the accelerator to the VM. This requires a full implementa-

tion of the GPU driver in the guest OS. It also requires hardware support, often not found on edge or mobile devices This technique makes the device API available inside the VM. As such, separate implementations are needed for different accelerator devices.

Instead, we use vAccel to provide a generic interface for XPU access in the VM. On the host, calls to vAccel are implemented using device-specific backends.

Challenge #4: Adaptability to different models/LoRAs. The model is a central component of LLM inference. Using different models based on the request's circumstances and the user's requirements can improve the overall experience. Similarly, selecting appropriate LoRA adapters is advantageous for receiving the best results.

Proprietary solutions restrict their implementation to a single statically served model. This simplifies logic, avoiding the need to handle multiple models simultaneously. Query scheduling does not need to take into account the model of any request.

Our approach is completely model-agnostic. Functionality we provide uses configurable policies to allow tuning for specific models. Additionally, large parts of the implementation automatically adjust to the model architecture. The model and LoRA can be selected for each request. A separate component handles the logic required to switch the active model and schedule requests in advantageous order.

4 Design

4.1 LLM Orchestration: Model and LoRA Selection

Adaptability to models and LoRAs. LROS provides a complete inference service to the device. As such, it needs to handle requests made from various sources. Due to the differing demands software has, a specialized model can significantly improve output quality. For example, using a model for text translation requires a different kind of answer than summarizing a single-language text. The applications know their workloads and can make informed decisions on the required model. To support this, the inference request contains a field that specifies a model.

LoRA adapters can adjust a model's behavior to support a wider range of tasks. This solution is lighter than choosing a fully specialized model. Therefore, it is promising to use them in conjunction with model specialization. In this way, multiple applications can share a base model, and each can choose a custom subset of LoRA adapters. An optional field in the request structure specifies the LoRA adapters to apply to the request.

Orchestrator. The orchestrator is the central component that coordinates the interaction of all other subsystems. The specific requirements of on-device inference, along with the implementation as a system service, necessitate such a component. The various applications that may access the inference service will send requests in an uncoordinated fashion. When two requests use the same model, their inferences can be batched to improve efficiency. Another task handled here is switching the model and loading the LoRA adapters. Any optimizations that target resource efficiency can be integrated here. The orchestrator is responsible for handling incoming requests and invoking the required actions.

Workflow. The basic workflow of this component (Figure 4.1) looks as follows. When a request arrives from the host operating system, it is placed into a queue (①). This allows LROS to handle multiple concurrent inference requests without assistance from the host. To schedule a new inference task, the controller parses the request to determine the required model and, if applicable, LoRA. If these are different from the currently loaded ones, it instructs the memory manager to load them (②). Then, the controller dequeues the request (③) and sends it to the inference engine.

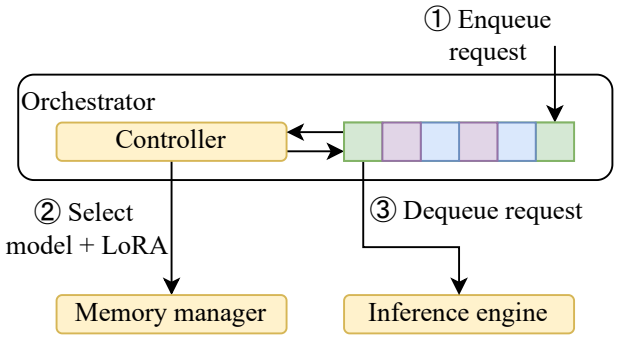


Figure 4.1: LROS orchestrator.

4.2 Lightweight Library OS

LLM inference in resource-constrained environments needs to deliver fast responses with high throughput while simultaneously keeping resources available for other processes. Preserving low latencies requires keeping the initialized inference engine loaded. Due to the high memory demands associated with the model and compute buffers, this blocks other memory-hungry processes. Similarly, high throughput is achieved by high CPU utilization, preventing other processes from executing. One solution to this tradeoff is to isolate the resource-hungry application using virtualization.

The software stack for running inference then includes the operating system. To best balance the tradeoff, it is vitally important to start with an operating system that, in itself, is optimized for low overheads. Traditional operating systems are general-purpose. For this reason, they carry functionality that is not applicable to LLM inference. This increases their memory footprint and introduces overhead.

Furthermore, it may be necessary to fully shut down the OS when the LLM inference service is not currently needed, allowing the host to reclaim any committed resources. Low response time is a crucial requirement for user-initiated queries. Therefore, it is important to ensure a fast boot time. In particular, heavyweight OSes are slow to boot, forcing designers to keep them running longer, wasting resources. Traditional operating systems fail in both of these regards. Stemming from their support for a wide variety of devices and systems, they exhibit long boot times and large memory footprints.

Therefore, we have decided to design LROS based on the Unikernel architecture. Unlike the design of regular general-purpose operating systems, which separate the execution environment into kernel space and user space, the application and operating system services run in the same execution context. This design philosophy has several

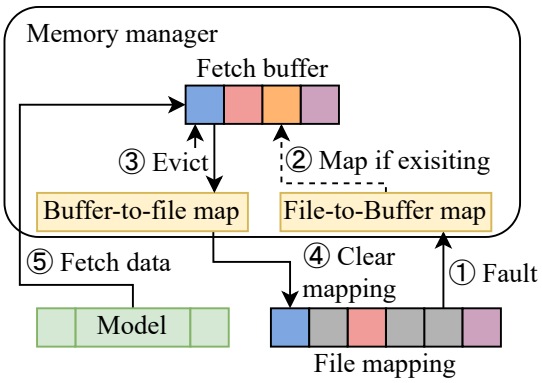


Figure 4.2: LROS memory manager.

Function	Description
mmap(...)	Creates mapping for file prefetch
setPrefetchPolicy(p)	Set policy for block prefetch
setEvictionPolicy(p)	Set policy for block eviction
setBlockSize(s)	Set size of prefetch blocks

Table 4.1: Memory Management API

advantages. First of all, it directly links the OS and application code into a single image, thus eliminating costly system calls that require context switches and replacing them with regular function calls. Additionally, this design makes application symbols available to the kernel and vice versa. The reduced barrier facilitates integration of application-specific interfaces. In particular, this allows us to easily introduce abstractions that inform the kernel components of requirements inherent to LLM inference.

General-purpose operating systems are designed for systems running concurrent processes, which introduces complications to many components. In contrast, unikernels enable the specialization of the OS to a single application. Moreover, the OS can be stripped of unnecessary functionalities at build time, producing a minimal system specialized to the application. This results in smaller images, lower boot times, and reduced overheads for any interaction with the kernel.

4.3 Memory Management

Context. LLM inference references large amounts of data in the form of the model weights. For optimal inference performance, the full model should be kept in memory at all times. This conflicts with the idea of conserving system resources. The application must fit within the externally set memory budget. In the extreme case, the available physical memory may not even be large enough to fit the full model. Keeping the model fully loaded in memory even when the user does not use it for a while is temporally inefficient. The memory is occupied by the model weights and cannot be used for any other process. On the other hand, keeping them loaded permanently allows computations to start immediately upon the arrival of the next request, without waiting to reload the model from disk. As a result, on-device LLM inference requires careful consideration of model weight memory management. In contrast, most inference systems only consider the case where the entire model is loaded once at the beginning and kept in memory for the lifetime of the program.

Model weights can be accessed in the program via file-backed memory mappings. Under memory pressure, the OS can transparently discard pages of file-based mappings that have not been modified. Limiting the memory available to the VM running the inference engine can therefore run inference with low memory usage.

Problems with existing interfaces. The ideal policy for page replacement is dependent on the architecture of the LLM model. For every output token, inference generally accesses all weights in a pre-determined order. Typically, model weights are stored in the file such that access order is optimized, i.e., weights needed for successive operations are placed at increasing addresses. Such dense linear reads can be indicated to the OS using the POSIX `madvise` call with the `MADV_SEQUENTIAL` flag. In contrast, composite models such as Mixture-of-Expert (MoE) models access only a portion of the model for each token. Other deviations may be caused by shared weights, e.g., the RoPE or the embedding tensors. General-purpose operating systems are oblivious to the semantics of memory on the userspace side. In particular, they are unaware of the model's memory layout or its unique access pattern. IO operations have fixed overheads that are the same regardless of the requested data size. This cost is exaggerated on virtual machines, where an expensive VMEXIT is needed for file access. Hence, it is suboptimal to load from the file at the granularity of the standard page size of 4 KiB.

Our solution. We propose a tensor-aware memory manager. This manager adapts its strategy by using page sizes tailored to the specific tensor and the overall model size. Furthermore, it employs a background thread to proactively prefetch model weights just before they are needed for inference.

The workflow and system interactions are shown in Figure 4.2. The orchestrator

maps the model file into memory using the `mmap` interface. This instructs the memory manager to start prefetching the file into a buffer. Accessing a new part of the file will cause a memory fault (①). The memory manager keeps track of which parts of the file are currently placed in its buffer. So it checks if the requested block resides in the fetch buffer. In that case, the pagetable is updated (②) to point to the correct part of the buffer. Otherwise, the memory manager determines a slot in the buffer that should hold the corresponding block from the file. When the buffer is already full, it must evict an existing slot (③). If that slot is referenced by the pagetable, it needs to clear this entry (④). Lastly, in step ⑤, it fetches the block from the file into the buffer. The memory manager will continue in the background, fetching more blocks from the file.

Using the API (Table 4.1), the application can specify custom prefetching policies on multiple dimensions. First, the inference engine can select the appropriate block size for its workload. Additionally, a customizable prefetching policy determines which block to retrieve next from the file. This enables adaptation to complex access patterns differing from the standard linear accesses. When the buffer is smaller than the model file, once it is full, blocks need to be evicted to make space for continued prefetching. The standard is the circular pattern, which replaces the earliest-fetched block. Instead, a custom policy selects the block to evict. In this way, it accommodates complex replacement patterns.

4.4 Accelerator Integration: vAccel

To enable accelerator access in our virtualized unikernel environment, we integrated vAccel. This abstraction provides a mechanism for generic access to accelerators independent of their hardware implementation. In general, virtualization introduces complications for accessing devices. The isolation prevents the standard pathways. Passing the device fully through to the VM requires hardware support and needs the complete device driver on the guest side. On the other hand, traditional API remoting approaches pose a high implementation hurdle, requiring support for fine-grained device APIs.

vAccel can be divided into multiple subcomponents that fulfil different roles in enabling the access of accelerator devices. The workflow is shown in Figure 4.3.

vAccel exposes an API to the application. This *user API* (Table 4.2) includes management functions, such as session initialization and release, and operations that correspond to actions executed on the accelerator device, such as `matmul` (①). In this way, the application communicates with the *device driver*. Since the VM does not have direct access to the host OS's devices and drivers, a separate driver must be implemented within the VM. It packages the application's API calls into a binary format that

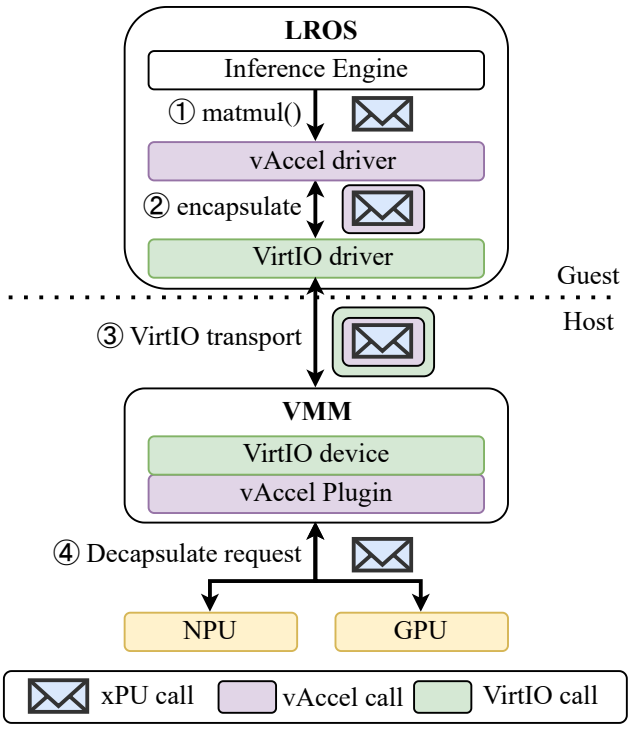


Figure 4.3: vAccel: The inference engine can access heterogeneous compute accelerators via a generic API.

can be transmitted to the host (②). The interface for communication to the host is the custom *vAccel PCI device*. We modified the VMM to support this custom device based on the VirtIO PCI interface, allowing the guest OS to efficiently relay requests (③). The hypervisor handles the host side of the connection. It unpackages the requests and forwards them to the *vAccel runtime* library for execution. Results are packaged and transmitted back via the same PCI interface.

The runtime accesses accelerators through device-specific *vAccel plugin* libraries. The user specifies the desired plugin at startup. During runtime initialization, the plugin is loaded. As a result, the program can be dynamically adjusted to its environment without requiring any changes to the source code or even a recompilation. The API design facilitates this by implementing generic operations (e.g., matrix multiplication) instead of device-specific APIs. The plugin provides implementation for the vAccel operations for a concrete device (④). The decoupling of application-side and device-side API makes the vAccel design a natural fit for the isolated nature of VMs.

With the presented design, using the PCI transport mechanism, vAccel can be

Management Functions	
sess_init(*flags*) → *s_id*	Initialize a session
sess_free(*s_id*)	Release session
Operations	
matmul_create(*s_id*, *info*) → *ctx*	Create a context for matrix multiplication
matmul_destroy(*s_id*, *ctx*)	Free matrix multiplication context
create_mem(*s_id*, *ctx*, *size*) → *t*	Create tensor memory
destroy_mem(*s_id*, *ctx*, *t*)	Destroy tensor memory
matmul_run(*s_id*, *ctx*)	Execute matrix multiplication context
matmul_set_io_mem(*s_id*, *ctx*, *t*, *role*)	Assign tensor memory to role of context
matmul_set_matrix(*s_id*, *t*, *src*, *n*)	Set tensor data
matmul_get_matrix(*s_id*, *dst*, *t*, *n*)	Get tensor data
matmul_get_props(*s_id*) → *props*	Get device properties

Table 4.2: vAccel API

viewed as API remoting. Crucially, it provides significant advantages over traditional methods. The exposed API is generic, allowing the development of a system suitable for the heterogeneous landscape of mobile and edge devices. Differences in hardware design and the available APIs can be easily handled without requiring changes to the application. Secondly, the API is coarse-grained. This reduces implementation effort on both the system and application sides by lowering the number of API calls required for any action.

5 Implementation

5.1 Unikraft

Our unikernel is based on the Unikraft operating system [Kue+21]. Due to the high degree of modularization in this library operating system, it is possible to provide a system containing exactly the components required and omit any unnecessary functionality, which can clutter your image. Application porting is simplified by providing a POSIX-compatible API. For the C standard library, there are multiple choices. The widely used glibc implementation does not fit into this scheme. It is large and monolithic. Instead, Unikraft includes a built-in `nolibc` that provides the most basic functions, such as `memcpy`. For more advanced projects, musl or newlib provides full-fledged implementations of the standard C library. We chose to base our implementation on musl. Since the llama.cpp inference engine is partly implemented in C++, we also had to include the libcxx library.

Unikraft, in its current implementation, only has very basic support for the simultaneous execution of multiple threads. Still, we had to enable threading, since llama.cpp has an execution model that splits work across threads, even when the thread pool worker count is set to one.

Most libraries work for our use case without modification or special configuration options. However, some minor tweaks are necessary to ensure smooth operation.

Scheduling. Scheduling assigns CPU time to runnable threads, allowing them to execute instructions. General-purpose operating systems need to balance the execution of a multitude of threads across a much smaller number of physical CPU cores. As a result, they use preemptive scheduling, which interrupts running threads at intervals and then schedules the next available thread.

Unikernels have only a limited, controlled number of threads. Therefore, they typically use a so-called cooperative scheduling approach. This means threads continue running without rescheduling until they voluntarily relinquish the execution right. The reasons for this include the thread being blocked, e.g., while waiting for an IO operation to complete, or by explicitly using the `sched_yield` function.

Applications often do not expect this behavior and, as a result, get stuck in infinite loops or encounter fairness issues. Thread coordination in llama.cpp uses condition variables as implemented in the C++ standard library. The worker threads wait for the

main thread to notify them that their queue contains new tasks. Under the hood, the implementation uses pthread, which in turn is based on the `futex` system call.

The expectation is that worker threads will pick up tasks from their queues soon after they are added. As this also includes background tasks, such as the output of partial results, the main thread will not actively wait for their completion. This results in big delays to these tasks.

To alleviate these issues to some degree, we activated a configuration option that prioritizes the newly created thread on `clone` system calls, thereby starting it before returning to the main thread. Similarly, we added a preference to the `futex_wake` system call, allowing woken threads to be executed by implicitly yielding execution before returning.

Memory allocation. The standard memory allocator included in Unikraft employs a straightforward strategy for partitioning the available memory into individual allocations returned to the program. In particular, this affects the management of the heap memory region and the access through `malloc` and `free`. They always immediately reserve the full region of memory required to satisfy the request. Standard implementations in other contexts typically handle large requests comprising multiple full memory pages differently from small ones. Instead of using the nominal program heap, they directly use `mmap` to satisfy the request. This has the advantage of not reserving memory pages until they are first touched.

The inference engine implementation allocates large buffers to hold the execution graph and intermediate activation tensors. Only small portions of these buffers are used under typical circumstances. Under Unikraft, the allocation strategy incurs significant overhead. This is especially relevant in the targeted scenario of memory-limited devices. We modified the implementation to use demand-paged anonymous memory mappings for allocations larger than 16 MB, mimicking the behavior of the `mmap` system call.

Cleanup. C++ structs can have destructors, which are automatically executed once an object goes out of scope. This is often used to release held resources or cleanup state associated with the struct. Globally accessible variables with so-called static storage duration are cleaned up when the program exits. llama.cpp relies on this behavior to flush output queues, among other cleanup actions. The Unikraft runtime implementation executes shutdown tasks only when the `exit` system call is explicitly called, not upon returning from the main function. Therefore, we modified all return points of the main function to call `exit` instead.

One further problem is that the default Unikraft configuration gets stuck in an infinite loop when shutdown is executed while multiple threads are still registered with the scheduler. Since the last threads are only terminated as part of the termination routine, this prevents a proper execution finish. We enabled the graceful shutdown

configuration option, resolving this issue by properly completing the secondary threads' shutdown procedure after the main thread exits.

5.2 Inference Engine: llama.cpp

We chose to base the implementation on the llama.cpp inference engine. It is an open-source project that supports a wide variety of LLM model architectures. A multitude of available backends support a diverse set of target devices or platforms, ranging from CUDA implementations for NVIDIA GPUs to optimized CPU-based implementations that use vectorization or BLAS libraries.

To support the use of accelerators using the vAccel-based interface, we created a custom backend that uses our generic operations to implement matrix multiplications. To reduce implementation complexity, only very specific operations are implemented using this backend. The remaining operations are executed by another backend. In particular, they will be executed using the fallback CPU backend.

5.3 vAccel

The integration of vAccel into our design required changes to multiple key components at all depths of the software stack.

Inference engine. The llama.cpp inference engine is the part of the system responsible for executing the calculations that make up the inference. To support different hardware platforms for inference, the calculations requested by the model architecture are represented using a hardware-agnostic computation graph. Each node represents a tensor involved in the inference, where the operation, type, and parameters determine the computation to perform. Backends are responsible for implementing computations for every node in the graph. Multiple backends can be loaded at the same time, allowing the engine to share the computations across them. This allows allocating the tensors so that data movement is reduced. Additionally, backends can selectively support some actions, potentially falling back to the standard CPU-based backend for operations without specialized, more efficient implementations.

Like typical NPU platforms, such as the Rockchip RK3588, we decided to limit the implementation to matrix multiplication. These account for the vast majority of operations and time spent at each inference step. Therefore, we enable high utilization while lowering the implementation effort and ensuring compatibility with common hardware. The implementation adapts an existing llama.cpp backend that uses the Rockchip RKNN matrix multiplication C API. We replaced all API calls with corresponding ones of the vAccel API Additionally, we made minor changes to make the implementation

more generic, allowing it to run on platforms with execution models different from the Rockchip one on which it is based. As such, we introduced additional API calls to access the tensor data allocated on the device rather than using direct memory accesses. We also modified the tensor layout transformation to the device-native form, to depend on device properties retrieved via a vAccel API call.

vAccel driver. The driver is part of the Unikraft library OS's driver subsystem. It handles communication with the hypervisor via a paravirtualized device based on the VirtIO PCI interface. At boot time, when the Unikraft PCI subsystem is initialized, the driver is also initialized. The vAccel device is discovered using standard PCI probing. If an appropriate device is found, the driver configures the communication channel's resources. It registers the device file with Unikraft, which is how the integrated runtime library interfaces with the driver. Then, it allocates memory for the Virtqueue used to transmit data to the hypervisor and sets it up. It configures the PCI device to use this queue via its control registers. It should be noted that the same queue is also used to receive data, since each receiving action is linked to a previous send, which simplifies handling.

During application execution, the driver handles the following tasks: First, it transmits requests it receives from the application to the hypervisor. Here, it has to convert between the vAccel request format and the layout required by the VirtIO protocol. Every input and output value requires an associated memory location. To reduce the hypervisor's implementation complexity, we impose the additional restriction that all buffers must be allocated using contiguous physical addresses. We ensure both of these requirements by allocating a separate kernel buffer, which we use to exchange the data by copying from (to) the passed buffers. The last step of transmission is enqueuing the buffers into the virtqueue.

Elements in virtqueues are specified by so-called descriptors, which include the length and physical address, along with a flag that distinguishes read-only and write-only buffers. Descriptors can be chained to be able to join multiple buffers into a single request. Every argument of the vAccel operation corresponds to a separate descriptor. In addition to the user-facing arguments, there are also some internal arguments. The first argument is the request header, which specifies the type of action (create/destroy a session or perform an operation), the session the request belongs to, and the number of input and output arguments. For each user argument, the buffer length is collected into an array, separated into input and output. These are transmitted as the second and third arguments. At the end of the argument list, the request status code is appended as an additional write-only argument.

After submission of the request, the driver waits for its completion. The thread that triggered the request is suspended until the response is received. The hypervisor

signals completion by triggering an interrupt request, which is then passed to a driver handler. This handler reads the status code from its buffer and wakes up the waiting thread. After waking up, the request is completed by copying the output arguments back into the user-supplied buffers. Lastly, the kernel buffers used to transmit the user arguments are freed.

Hypervisor. Our hypervisor of choice for running the LROS is QEMU. We use KVM for fast, efficient virtualization. The integration of vAccel into our design requires establishing a communication channel between the host and guest operating systems to relay the requests and their corresponding responses. The basis of this communication is the VirtIO PCI interface.

We implemented a custom VirtIO device type, which represents the semantics of the vAccel transport protocol. In addition to the already described driver, which is part of the guest OS, a device implementation in the hypervisor is needed. The QEMU implementation is based on the VirtIO crypto device design. As such, the implementation is split into a backend that interfaces with the host vAccel runtime library and a VirtIO device that interfaces with the guest using the VirtIO protocol. The information placed into the virtqueue by the guest OS is unpacked again into the same layout it had before the driver placed it in the queue. QEMU already supports different kinds of VirtIO devices, so the existing infrastructure simplifies the adaptation. Notably, reading and decoding the descriptor chain are already implemented.

So QEMU loads the request header to get the action type, the session ID, and the number of input and output arguments. For security reasons, pointers to memory regions assigned to the guest VM are not given to external code. So the arguments are copied once more into newly allocated buffers, according to the lengths specified by the arrays in the second and third virtqueue descriptors. A newly allocated array holds structs that specify the length and address of these buffers, so it can be passed to the vAccel runtime.

The backend uses the decoded information to call the correct vAccel API function, passing the session ID, the created input and output argument arrays, and their count. The API call returns a value indicating whether an error occurred during execution, which is stored in the implicit status argument's buffer. Afterward, the contents of the output buffers are copied back to the corresponding buffers received via the virtqueue. The output buffers are added back to the virtqueue in the section where they are returned to the VM. To signal the guest OS that the operation is complete, QEMU triggers the device's IRQ line. All allocated temporary buffers are freed before returning.

vAccel runtime library. The runtime library is a central piece in the vAccel software stack. It is a regular userspace library that runs on the host. It orchestrates access to

accelerator functions via dynamically loaded plugins.

The hypervisor links to this library, so any actions run in isolation from and independently of the unikernel guest operating system. When the hypervisor starts, the library is initialized. Using the `VACCEL_PLUGINS` environment variable, the user specifies a colon-separated list of shared library paths. These shared libraries provide plugins for vAccel that implement its API operations.

At initialization, they are loaded using `dlopen` and the special `vaccel_plugin` symbol, which must be included in every plugin whose information is acquired. Then, the plugins are initialized using the function provided in their information symbol. Here, a plugin can register implementations for any of the operations included in the vAccel API. When multiple plugins provide implementations, they are stored in a linked list.

Interaction between the downstream application and vAccel is based on sessions. Sessions are identified by a unique ID assigned at session creation. During session initialization, the program can supply flags indicating a preference for specific plugin types (e.g., CPU or GPU). Choosing the appropriate implementation prefers using operation implementations from plugins whose type matches the session preference.

5.4 vAccel Operations

To efficiently support inference using vAccel's virtualization mechanism, we added specific API operations to enable matrix multiplication. Since our llama.cpp backend implementation is based on a Rockchip NPU for matrix multiplication, the operations we settled on resemble the Rockchip RKNN C API. Therefore, the implementation for the OrangePi 5 Ultra, which uses the Rockchip RK3588 NPU, primarily consists of wrappers around the native API. The implementation for the NVIDIA Jetson AGX Orin maps operations to the appropriate CUDA API calls.

In the following, I will list and explain the operations in detail:

- `MATMUL_CREATE`: Creates a matrix multiplication kernel. This operation initializes a context for matrix multiplication of specific matrix dimensions (M, N, K for multiplying $M \times N$ and $N \times K$ matrices). Additionally, it is required to pass the matrix-multiplication type, which can be either the multiplication of 16-bit floats with a 32-bit float result or the multiplication of 8-bit integers with a 32-bit integer result. Another required parameter is the matrix layout, which can be either the standard row-major representation or a custom block-wise rearrangement used by Rockchip NPUs internally to optimize memory access patterns. Using the native layout directly increases performance. The return values of this operation are a handle to the created matrix-multiplication kernel and a structure containing the sizes and dimensions of the input tensors. This wraps the `rknn_matmul_create`

API call. The CUDA implementation simply validates the parameters and saves them to a buffer; the returned handle is merely an index into the buffer.

- `CREATE_MEM`: Allocates memory for a tensor. This operation allocates memory accessible on the device. It takes as parameters the kernel handle the tensor belongs to and the size of the memory to allocate. Its return value is a handle of the allocated memory. This wraps the `rknn_create_mem` API call for the NPU. For the GPU CUDA implementation, it is a simple wrapper around the `cudaMalloc` call.

- `DESTROY_MEM`: Deallocates memory for a tensor. This operation takes the handle of a memory block allocated using the `CREATE_MEM` operation and frees it. An additional parameter is the handle of a matrix kernel. On the RKNN API, this maps to the `rknn_destroy_mem` operation. The CUDA implementation just calls `cudaFree`.

- `MATMUL_DESTROY`: Destroys a matrix multiplication kernel. This operation releases the context and related resources of a given matrix multiplication handle. The RKNN implementation just wraps the corresponding `rknn_matmul_destroy` call. On the CUDA backend, this operation just marks the corresponding array index as unused.

- `MATMUL_SET_IO`: Sets the input and output tensor locations of a matrix multiplication kernel. Matrix kernels do not have set input or output buffers. Instead, tensors allocated using the `CREATE_MEM` operation must be assigned to these roles before execution. To identify the kernel, this operation takes the handle of the handle. The tensor to assign is specified by its handle, and the role of the tensor is specified by passing the tensor attributes received as a return value of `MATMUL_CREATE`. The NPU backend just forwards this call to `rknn_set_io_mem`. This call can potentially be expensive, especially when the used matrix layouts of input tensors are not the performance-oriented, rearranged ones. In this case, the data may be copied to additional internal buffers for rearrangement. The CUDA implementation just saves the handle to the internal kernel context.

- `MATMUL_SET_CORE_MASK`: Sets a mask that defines which accelerator cores the matrix kernel can run on. The Rockchip NPU has three cores. It is required to assign kernels to all cores equally to achieve optimal performance. This operation allows overriding the default behavior, which randomly chooses the core for execution. The kernel is specified by its handle. A second parameter specifies the mask to be set. Accordingly, the NPU implementation maps this directly to

`rknn_matmul_set_core_mask`. NVIDIA GPUs do not have an equivalent concept of cores. Therefore, the operation is a noop in this implementation.

- `MATMUL_RUN`: Runs a matrix multiplication kernel. Given the handle obtained using `MATMUL_CREATE`, this operation starts the execution of the corresponding matrix multiplication and synchronously waits for the operation to finish. Input and output data are specified by preceding calls of `MATMUL_SET_IO`. The type of the matrix elements and the dimensions of the input and output matrices are specified at the creation of the handle. For the Rockchip implementation, the corresponding action is a call to `rknn_matmul_run`. On CUDA, the backend provides this operation using cuBLAS and its `cublasGemmEx` function. The definition of matrix multiplication as part of the GGML library in llama.cpp is unusual: $C = \text{mul_mat}(A, B)$ means $C^T = A \cdot B^T$, or equivalently, $C = B^T \cdot A$. Due to these peculiarities, the call does not have the expected parameter ordering. Instead, the A and B tensors are switched. For the first matrix (B), we select the transposition operator. The parameters specifying the matrix sizes are reordered accordingly. The NPU implementation does not require these modifications because the transposition is resolved on the rearrangement to the efficiently repacked native matrix layout.

- `MATMUL_SET_MATRIX/MATMUL_GET_MATRIX`: Accesses the data of a matrix. The memory of tensors allocated using the `CREATE_MEM` interface is not necessarily directly writeable from the call context of the operation. Additionally, the isolation introduced by execution in a VM presents another hurdle for access. Therefore, we added a generic operation that unifies access to the matrix data. The parameters correspond to the standard interface for memory copying: a host buffer pointer, a device tensor handle, and the number of bytes to copy. The NPU implementation simply uses `memcpy` to copy between the host buffer and the DMA memory's virtual address region. On the GPU, a call to `cudaMemcpy` has the correct semantics.

- `MATMUL_GET_PROPS`: Retrieves properties related to the backend implementation of the matrix multiplication. Currently, the only available property is whether repackaged matrix representations are supported. The parameters are a buffer to hold property values and the buffer's size in bytes. Corresponding to the hardware capabilities, RKNN supports the alternative matrix representation, whereas CUDA does not.

6 Evaluation

6.1 Experiment Setup

Hardware platforms. LROS was designed with two deployment scenarios in mind: Edge and mobile devices. To reflect this decision, we evaluate our implementation on two different hardware platforms. The OrangePi 5 Ultra is a mobile platform. It is powered by the Rockchip RK3588 SoC that includes an 8-core Arm CPU comprising 4 high-performance Cortex-A76 and 4 high-efficiency Cortex-A55 cores. To accelerate machine learning tasks, it includes an NPU that supports Int4, Int8, and Float16 computations. It contains 16GB of RAM. Secondly, we evaluate based on the NVIDIA Jetson Orin AGX. This device is more powerful and is intended for edge computing scenarios. It contains a 12-core Cortex-A78AE CPU. Alongside this, a 2048-core NVIDIA Ampere GPU with 64 Tensor cores is available for accelerated computations. The 64GB of RAM is shared between the CPU and the GPU.

6.2 Light-weight OS

Research Question. Does LROS provide a light-weight operating system?

Metrics. We measure the time from boot to when the inference engine starts executing. Another important metric is the kernel's size. It can be quantified as the size of the image file stored on disk or the kernel's memory footprint once it has been loaded.

Methodology. Microbenchmarks are used to assess these basic system properties, including complexity, overheads, and footprint.

To measure the operating system's boot time, we inserted instructions into the kernel's startup code that allow us to record timestamps at various points in the startup process. We adapt the process described in [Gar19] to work on the Arm architecture. Since Arm does not provide port-based IO, we instead use the `hvc` instruction, which similarly causes a traceable event on the host. We identify our events using the immediate of the instruction and the first argument placed in `r0`. We take care to use only `r0` values that do not decode to valid hypervisor calls in KVM, to avoid influencing the execution state.

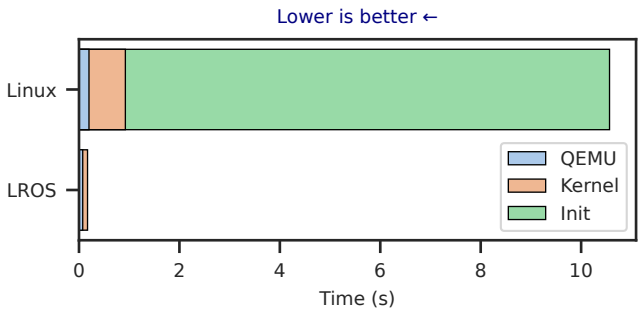


Figure 6.1: Boot time of the VM.

Using the bpftrace tool, we obtain timestamps of different points of the execution. This allows us to report different time measurements:

- Hypervisor initialization (QEMU): Time between the start of the QEMU `main` function and the first call of `ioctl(KVM_RUN)`
- Kernel initialization (Kernel): Time until the kernel passes control to user space. On Linux, this is `init_start`, for LROS we insert the tracing just before calling the `main` function of the application.
- Userspace initialization (Init): Time until systemd has dispatched all services. This is not relevant for LROS, since there is no user space in the unikernel.

The image sizes of the unikernel and traditional operating systems are not equal. While the LROS unikernel is self-contained and can be executed standalone, the Linux kernel provides only operating system-level functionality, and a file system is needed to provide other basic functionality. So we added the kernel size and the size of a basic Linux drive image.

The memory footprint of the different systems is a dynamic quantity that is highly dependent on the OS's actual usage pattern. The workflow used for measurement was to run the inference engine, pass an argument that just prints usage information, and then shut down the OS. The numbers are obtained by checking how much of the memory mapping assigned to the VM using `ioctl(KVM_SET_USER_MEMORY_REGION)` has been touched at the hypervisor's exit. For consistent values, we average 5 executions.

Baseline. The baseline is a Linux VM created with NixOS. The kernel version used is 6.16.9.

Results. The boot time (Figure 6.1) of LROS is just 0.17s, which is 62× lower than the time needed to fully boot the Linux VM. It may be possible to reduce user-space initialization in the Linux system, but even when you consider only kernel time, there

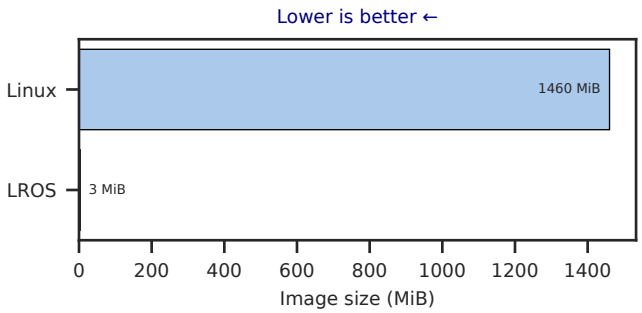


Figure 6.2: Size of the disk image.

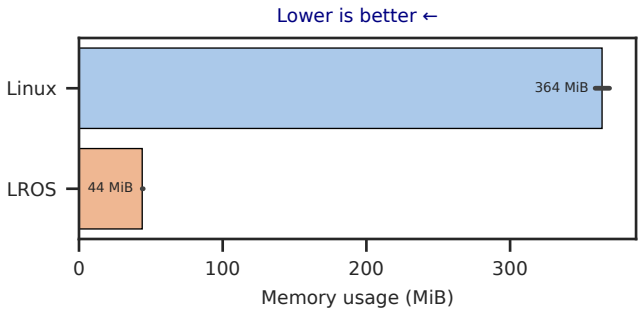


Figure 6.3: Amount of memory touched during system boot and inference engine startup.

is still a 5.4× advantage for LROS. The image sizes (Figure 6.2) also show a clear advantage for LROS over the NixOS Linux instance. The memory footprint (Figure 6.3) also shows a significant advantage for LROS (8.3× lower memory usage). In particular, the Linux system's lower memory usage than disk size indicates that the disk image contains superfluous files that are never loaded.

6.3 OS Specializations

Research Question. Can the OS Specializations implemented in LROS provide improved inference performance?

Methodology. We evaluate the effectiveness of the custom memory prefetcher we implemented. Memory prefetching is intended to efficiently run inference under memory constraints. To show its effectiveness at preserving performance even under memory pressure, we test two scenarios. First, we test with a memory allocation for the VM that is large enough to fit the entire model at once. We also test by limiting the

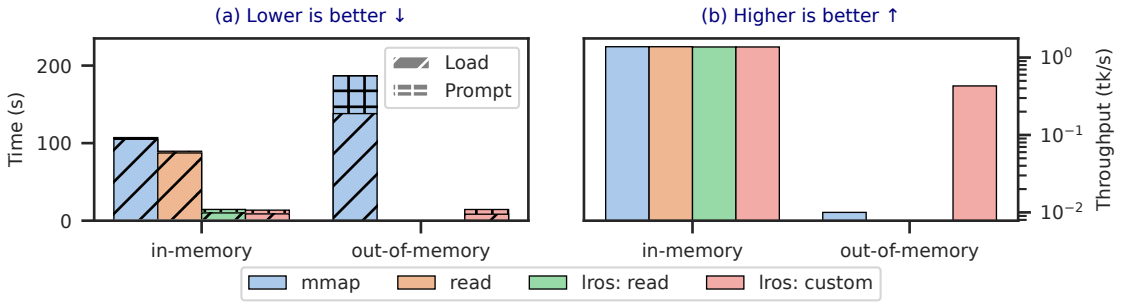


Figure 6.4: Performance under different model loading strategies: (a) time until first token (TTFT) and (b) text generation throughput. LROS *offers significantly improved latencies until the first token is processed. The performance on both metrics degrades less than the baseline when the model only partially fits into memory.*

memory assigned to the VM to less than the total size of the model, which will cause eviction of the weights.

Metrics. We record the time it takes for the inference engine to load the model, the time required to process a prompt (pre-fill phase), and, in total, the time to first token (TTFT) for a cold start. We also measure the throughput achieved afterwards for generating new tokens (decode phase).

Benchmark. We run inference using the llama-batched-bench example application, which measures the time taken for inference with a given number of random input tokens and a specified number of output tokens. We chose to test using 16 input tokens and 8 output tokens.

Baseline. We compare the times recorded with LROS using the prefetcher to those with llama.cpp in a standard Linux VM. For the baseline, we use different options in llama.cpp that influence how the model is loaded. We measure using the default mmap-based model loading included in llama.cpp. Additionally, for in-memory inference, we include loading the model via a read-based interface. This can only work if all loaded tensors fit in memory, since, unlike file mappings, which can be paged out and reloaded, these need to remain permanently in memory.

Results. Memory prefetching saves significant runtime. Figure 6.4 shows the inference performance of various strategies for model loading. Depending on the configuration used, the entire model may be read in the load phase or only parts of it. Therefore, the only fair comparison is to combine loading and prompt processing into a single measurement and compare them together.

If the full model fits into memory, TTFT, including model loading via the custom

memory loader in LROS, is 87.31% lower than using the standard `mmap`-based interface and 84.83% lower than using `read`. When the model does not fit into memory, this gap grows to 92.28%.

The inference throughput achieved by LROS is within the measurement accuracy of the baseline when the model fits within the available memory. In the memory-constrained scenario, the performance using the prefetcher is vastly better. LROS offers a 32.8× higher throughput than the `mmap` based standard interface. The performance gains stem from two facts. LROS overlaps computation with prefetching of the next needed weights. Additionally, the block size of the file reads has been adjusted to better utilize the available bandwidth on the virtual filesystem.

6.4 Accelerator Integration: vAccel

Research Question. Can LROS achieve better performance by using accelerator devices?

Methodology. We evaluate the implementation of accelerator access using the vAccel interface.

Metrics. The two main metrics for quantifying inference performance are the time to first token (TTFT), which measures the latency of the highly parallel pre-fill phase, and the throughput, or token generation rate, of the decode phase.

Benchmark. We use the llama-batched-bench example application to run inference requests. Given a specific number of input and output tokens and a batch size, it generates random input tokens and runs inference tasks for all combinations. For this evaluation, we chose to test with 16 and 32 input tokens, 128 and 256 output tokens, and batch sizes of 1 and 4.

Baseline. We compare the performance numbers of LROS with the vAccel backend to those of LROS running purely on the CPU. For additional comparison, we run the CPU implementation within a Linux VM. The last comparison is running llama.cpp as a Linux process on the host using the full CUDA backend. This provides an upper bound of performance achievable with an optimized backend implementation.

Results. The performance of LROS with the CPU backend is very similar to that of llama.cpp in a Linux VM, which is also limited to CPU-only execution. This is true for inference latency and throughput. There are only negligible differences.

Inference latency or TTFT (Figure 6.5) shows no significant advantage when using the vAccel implementation. This stems from limitations in the backend implementation. Currently, the vAccel backend cannot be used for the prefill phase. It is therefore expected to obtain the CPU backend's performance, which provides the fallback implementation. On the other hand, the full CUDA implementation on the host

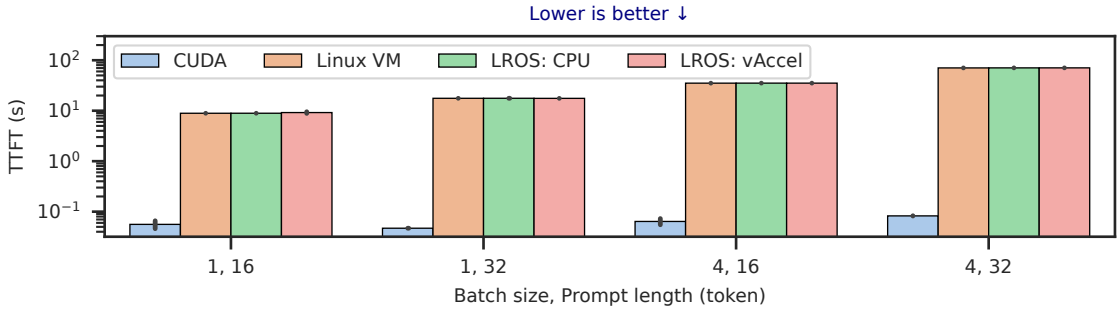


Figure 6.5: Inference performance comparison: Time to first token (TTFT). The latency of inference operations is comparable to that of CPU-based inference.

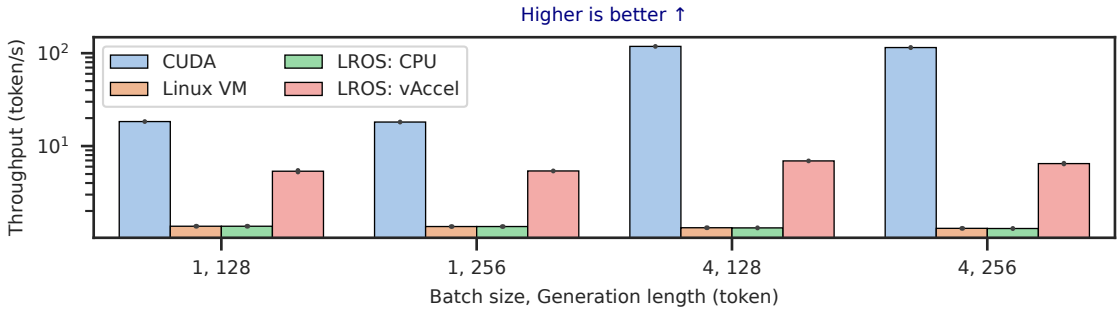


Figure 6.6: Inference performance comparison: Throughput. The token generation rate in the decode phase is significantly higher with vAccel than with CPU inference.

achieves significantly reduced TTFTs, indicating that substantial improvements in this regard can be achieved through broader adoption.

Inference throughput (Figure 6.6) shows a different picture. The vAccel implementation offers significant improvements compared to CPU-based implementations. The throughput is 4.0× higher for a batch size of 1 and increases to 5.2× for a batch size of 4. Due to frequent data transfers and the implementation of only the most prevalent instructions on the accelerators, the vAccel performance remains significantly lower than that of the full CUDA Linux process. It reaches 3.4× and 17× higher performance than LROS using vAccel for batch sizes 1 and 4, respectively. Adding a data path that reduces memory copies and expanding the number of computations the device can perform could therefore offer significant improvements.

7 Related Work

Resource-efficient on-device LLMs. Several projects have explored various methods to adapt models to enhance resource efficiency. Model compression techniques, such as pruning or quantization [Xu+21; Liu+21; Lin+24], can reduce memory requirements. These techniques are largely orthogonal to our project. They can be applied to the model before it is loaded into our system.

Other projects have devised algorithmic improvements that optimize inference under resource constraints. Hybrid schemes can be used to offload more demanding queries to servers [Ban+24]. Selective token processing [Jia+24] reduces load by focusing computation on important parts of the prompt. Improvements in memory access and transfer patterns [Ali+24] can significantly increase performance. Our memory prefetching customization achieves speedups in similar ways. They go further by exploiting model-specific information, which can be integrated into our approach by supplying suitable policies. We focus on improvements that are applicable without impacting the inference algorithms; these orthogonal improvements can be applied on top.

Various studies [Hus+25; Las+24] explore the performance implications of on-device inference. They highlight major challenges in balancing performance, energy efficiency, and accuracy under resource constraints. Their findings reinforce the need for specialized systems.

XPU for LLM inference. The potential of using NPUs and GPUs for LLM inference has led to numerous works exploring their use. `llm.npu` [Xu+25] considers the specific characteristics of NPUs. They offer unique opportunities and restrictions when used. `HeteroInfer` [Che+25] uses GPU-NPU parallelism to exploit all available compute devices. `PowerInfer` [Son+24] explores a hybrid scheme between CPU and GPU to handle scenarios where the model exceeds GPU memory. `PowerInfer2` [Xue+24] extends this approach to mobile platforms and NPU computations. All of these approaches leverage direct hardware access. Our focus on hardware isolation introduces unique challenges in accessing XPUs that are not accounted for in these approaches. Our generic implementation using vAccel provides a natural point for integrating hybrid schemes as they were presented.

Virtualization for on-device environments. `Hermes` [KB18] explores the use of a hypervisor for embedded microcontrollers. They discover several challenges associated

with virtualization on platforms lacking a memory management unit. Due to the high computational cost of LLM inference, we cannot run on similar platforms. VMWare implemented a hypervisor for mobile devices [Bar+10]. It focuses on supporting a second full operating system on a single smartphone. Their approach is intended to enable two separated, isolated environments without requiring a second device. The primary goal is to enhance security between these two domains. Another study [CC07] also found security as a major application. Our virtualization use case, which focuses on resource efficiency and performance isolation, is different. Nonetheless, security is one possible future expansion point. `Bao` [Mar+20] uses a static resource partitioning scheme to achieve strong isolation. Their scheme runs a bare-metal hypervisor directly on the hardware. The restrictions introduced by this implementation are incompatible with the LROS approach. In particular, vAccel relies on an underlying general-purpose operating system for executing the backend implementations.

Specialized OS using the unikernel architecture. Using unikernels to supply specialized functionality to user applications is a tested approach. `DiLOS` [Yoo+23] implements custom paging algorithms for disaggregated memory. This use case is similar to the memory prefetching from disk storage implemented in LROS. They also allow policies to tune the prefetch order. As a major limitation, their policies are independent of the application's execution and can only optimize based on page content, not program state. Unikernels have also been tested for database systems [LD24]. The extensive I/O requirements of databases impose even greater limitations than for LLMs. The Function-as-a-Service execution model is a promising candidate for unikernels [MPB24]. This model runs many short-lived, isolated tasks. They found significant advantages in startup latency, which supports our evaluation. `MorphOS` [Oke+25] extends unikernels with a mechanism for dynamically loading eBPF programs, enabling on-the-fly reconfiguration.

8 Conclusion

In conclusion, LROS provides a standalone end-to-end system for LLM inference in on-device contexts. The provided service runs in a virtualized environment to allow strict enforcement of resource limits and isolate its performance from concurrent tasks. We use the unikernel architecture to provide a light-weight, low-overhead system image. The strong coupling between the inference engine and the operating system enables optimizations, like a tensor-aware model weight prefetcher. Our design also considers integrating XPUs to accelerate computations via vAccel's generic, flexible interface.

One possible direction for future work is security. The model weights are a proprietary resource that model vendors invest significant time and effort in developing and training. To protect this advantage, it is paramount to prevent any leaks. For this purpose, the model needs to be protected on disk and during transmission using encryption. Additionally, it is important not to expose the weights loaded into memory during inference. Novel confidential VM technologies, such as Arm CCA, provide hardware-enforced memory isolation for the processes they contain. Central design decisions of LROS support its adoption. Virtualization is a requirement to exploit these mechanisms. The unikernel architecture significantly reduces code size and, in turn, the trusted computing base compared to a Linux deployment.

List of Figures

3.1 LROS overview. LROS *composes the Inference Engine and the specialized libOS into a lightweight unikernel.* 12

4.1 LROS orchestrator. 17
4.2 LROS memory manager. 18
4.3 vAccel: The inference engine can access heterogeneous compute accelerators via a generic API. 21

6.1 Boot time of the VM. 32
6.2 Size of the disk image. 33
6.3 Amount of memory touched during system boot and inference engine startup. 33
6.4 Performance under different model loading strategies: (a) time until first token (TTFT) and (b) text generation throughput. LROS *offers significantly improved latencies until the first token is processed. The performance on both metrics degrades less than the baseline when the model only partially fits into memory.* 34
6.5 Inference performance comparison: Time to first token (TTFT). The latency of inference operations is comparable to that of CPU-based inference. 36
6.6 Inference performance comparison: Throughput. The token generation rate in the decode phase is significantly higher with vAccel than with CPU inference. 36

List of Tables

3.1 LROS system API 13

4.1 Memory Management API 18
4.2 vAccel API 22

Bibliography

[Ali+24] K. Alizadeh, S. I. Mirzadeh, D. Belenko, S. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar. "LLM in a flash: Efficient Large Language Model Inference with Limited Memory." In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by L.-W. Ku, A. Martins, and V. Srikumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 12562–12584. DOI: 10.18653/v1/2024.acl-long.678.

[App26] Apple Inc. *CoreML Framework Documentation*. https://developer.apple.com/documentation/coreml. Accessed: 2026-02-02. 2026.

[Ban+24] J. Bang, J. Lee, K. Shim, S. Yang, and S. Chang. "Crayon: Customized On-Device LLM via Instant Adapter Blending and Edge-Server Hybrid Inference." In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by L.-W. Ku, A. Martins, and V. Srikumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 3720–3731. DOI: 10.18653/v1/2024.acl-long.204.

[Bar+10] K. Barr, P. Bungale, S. Deasy, V. Gyuris, P. Hung, C. Newell, H. Tuch, and B. Zoppis. "The VMware mobile virtualization platform: is that a hypervisor in your pocket?" In: *SIGOPS Oper. Syst. Rev.* 44.4 (Dec. 2010), pp. 124–135. ISSN: 0163-5980. DOI: 10.1145/1899928.1899945.

[CC07] L. P. Cox and P. M. Chen. "Pocket Hypervisors: Opportunities and Challenges." In: *Eighth IEEE Workshop on Mobile Computing Systems and Applications*. 2007, pp. 46–50. DOI: 10.1109/HotMobile.2007.20.

[Che+25] L. Chen, D. Feng, E. Feng, Y. Wang, R. Zhao, Y. Xia, P. Xu, and H. Chen. "Characterizing Mobile SoC for Accelerating Heterogeneous LLM Inference." In: *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. SOSP '25. Lotte Hotel World, Seoul, Republic of Korea: Association for Computing Machinery, 2025, pp. 359–374. ISBN: 9798400718700. DOI: 10.1145/3731569.3764808.

[Fra+23] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*. 2023. arXiv: 2210.17323 [cs.LG].

[Gar19] S. Garzarella. *How to measure the boot time of a Linux VM with QEMU/KVM*. https://stefano-garzarella.github.io/posts/2019-08-24-qemu-linux-boot-time/. Accessed: 2026-01-02. Aug. 2019.

[Goo26] Google. *Gemini Nano*. https://developer.android.com/ai/gemini-nano. Accessed: 2026-02-02. 2026.

[Gra+24] A. Grattafiori, A. Dubey, A. Jauhri, et al. "The Llama 3 Herd of Models." In: *arXiv e-prints*, arXiv:2407.21783 (July 2024), arXiv:2407.21783. DOI: 10.48550/arXiv.2407.21783. arXiv: 2407.21783 [cs.AI].

[Gun+24] T. Gunter, Z. Wang, C. Wang, et al. *Apple Intelligence Foundation Language Models*. 2024.

[Han+15] S. Han, J. Pool, J. Tran, and W. Dally. "Learning both Weights and Connections for Efficient Neural Network." In: *Advances in Neural Information Processing Systems*. Ed. by C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett. Vol. 28. Curran Associates, Inc., 2015.

[HMD16] S. Han, H. Mao, and W. J. Dally. *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding*. 2016. arXiv: 1510.00149 [cs.CV].

[Hus+25] E. J. Husom, A. Goknil, M. Astekin, L. K. Shar, A. KÃ¥sen, S. Sen, B. A. Mithassel, and A. Soylu. "Sustainable LLM Inference for Edge AI: Evaluating Quantized LLMs for Energy Efficiency, Output Accuracy, and Inference Latency." In: *ACM Trans. Internet Things* 6.4 (Nov. 2025). DOI: 10.1145/3767742.

[HVD15] G. Hinton, O. Vinyals, and J. Dean. *Distilling the Knowledge in a Neural Network*. 2015. arXiv: 1503.02531 [stat.ML].

[Jac+18] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference." In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2018.

[Jia+24] Y. Jiang, H. Wang, L. Xie, H. Zhao, C. Zhang, H. Qian, and J. C. Lui. "D-LLM: A Token Adaptive Computing Resource Allocation Strategy for Large Language Models." In: *Advances in Neural Information Processing Systems*. Ed. by A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak,

and C. Zhang. Vol. 37. Curran Associates, Inc., 2024, pp. 1725–1749. DOI: 10.52202/079017-0055.

[KB18] N. Klingensmith and S. Banerjee. "Hermes: A Real Time Hypervisor for Mobile and IoT Systems." In: *Proceedings of the 19th International Workshop on Mobile Computing Systems & Applications*. HotMobile '18. Tempe, Arizona, USA: Association for Computing Machinery, 2018, pp. 101–106. ISBN: 9781450356305. DOI: 10.1145/3177102.3177103.

[Kue+21] S. Kuenzer, V.-A. Bădoiu, H. Lefeuvre, S. Santhanam, A. Jung, G. Gain, C. Soldani, C. Lupu, Ş. Teodorescu, C. Răducanu, C. Banu, L. Mathy, R. Deaconescu, C. Raiciu, and F. Huici. "Unikraft: fast, specialized unikernels the easy way." In: *Proceedings of the Sixteenth European Conference on Computer Systems*. EuroSys '21. ACM, Apr. 2021, pp. 376–394. DOI: 10.1145/3447786.3456248.

[Las+24] S. Laskaridis, K. Katevas, L. Minto, and H. Haddadi. "MELTing Point: Mobile Evaluation of Language Transformers." In: *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. ACM MobiCom '24. Washington D.C., DC, USA: Association for Computing Machinery, 2024, pp. 890–907. ISBN: 9798400704895. DOI: 10.1145/3636534.3690668.

[LD24] V. Leis and C. Dietrich. "Cloud-Native Database Systems and Unikernels: Reimagining OS Abstractions for Modern Hardware." In: *Proc. VLDB Endow.* 17.8 (Apr. 2024), pp. 2115–2122. ISSN: 2150-8097. DOI: 10.14778/3659437.3659462.

[Li+24a] B. Li, Y. Jiang, V. Gadepally, and D. Tiwari. "LLM Inference Serving: Survey of Recent Advances and Opportunities." In: *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. 2024, pp. 1–8. DOI: 10.1109/HPEC62836.2024.10938426.

[Li+24b] X. Li, Z. Lu, D. Cai, X. Ma, and M. Xu. "Large Language Models on Mobile Devices: Measurements, Analysis, and Insights." In: *Proceedings of the Workshop on Edge and Mobile Foundation Models*. EdgeFM '24. Minato-ku, Tokyo, Japan: Association for Computing Machinery, 2024, pp. 1–6. ISBN: 9798400706639. DOI: 10.1145/3662006.3662059.

[Lin+24] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han. "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration." In: *Proceedings of Machine Learning and Systems*. Ed. by P. Gibbons, G. Pekhimenko, and C. D. Sa. Vol. 6. 2024, pp. 87–100.

[Liu+21] Z. Liu, F. Li, G. Li, and J. Cheng. "EBERT: Efficient BERT Inference with Dynamic Structured Pruning." In: *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. Ed. by C. Zong, F. Xia, W. Li, and R. Navigli. Online: Association for Computational Linguistics, Aug. 2021, pp. 4814–4823. DOI: 10.18653/v1/2021.findings-acl.425.

[Mar+20] J. Martins, A. Tavares, M. Solieri, M. Bertogna, and S. Pinto. "Bao: A Lightweight Static Partitioning Hypervisor for Modern Multi-Core Embedded Systems." In: *Workshop on Next Generation Real-Time Embedded Systems (NG-RES 2020)*. Ed. by M. Bertogna and F. Terraneo. Vol. 77. Open Access Series in Informatics (OASIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, 3:1–3:14. ISBN: 978-3-95977-136-8. DOI: 10.4230/OASIcs.NG-RES.2020.3.

[MPB24] F. Moebius, T. Pfandzelter, and D. Bermbach. "Are Unikernels Ready for Serverless on the Edge?" In: *2024 IEEE International Conference on Cloud Engineering (IC2E)*. 2024, pp. 133–143. DOI: 10.1109/IC2E61754.2024.00022.

[Oke+25] P. Okelmann, I. Meignan-Masson, M. Misono, and P. Bhatotia. "MorphOS: An Extensible Networked Operating System." In: *Proc. ACM Netw.* 3.CoNEXT4 (Nov. 2025). DOI: 10.1145/3768977.

[Pop+22] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean. *Efficiently Scaling Transformer Inference*. 2022. arXiv: 2211.05102 [cs.LG].

[Son+24] Y. Song, Z. Mi, H. Xie, and H. Chen. "PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU." In: *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. SOSP '24. Austin, TX, USA: Association for Computing Machinery, 2024, pp. 590–606. ISBN: 9798400712517. DOI: 10.1145/3694715.3695964.

[Vas+17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. "Attention is all you need." In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS'17. Long Beach, California, USA: Curran Associates Inc., 2017, pp. 6000–6010. ISBN: 9781510860964.

[Xu+21] D. Xu, I. E.-H. Yen, J. Zhao, and Z. Xiao. "Rethinking Network Pruning – under the Pre-train and Fine-tune Paradigm." In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Ed. by K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, and Y. Zhou. Online: Association for Computational

Linguistics, June 2021, pp. 2376–2382. DOI: 10.18653/v1/2021.naacl-main.188.

[Xu+23] D. Xu, W. Yin, X. Jin, Y. Zhang, S. Wei, M. Xu, and X. Liu. *LLMCad: Fast and Scalable On-device Large Language Model Inference*. 2023. arXiv: 2309.04255 [cs.NI].

[Xu+25] D. Xu, H. Zhang, L. Yang, R. Liu, G. Huang, M. Xu, and X. Liu. "Fast On-device LLM Inference with NPUs." In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. ASPLOS '25. Rotterdam, Netherlands: Association for Computing Machinery, 2025, pp. 445–462. ISBN: 9798400706981. DOI: 10.1145/3669940.3707239.

[Xue+24] Z. Xue, Y. Song, Z. Mi, X. Zheng, Y. Xia, and H. Chen. *PowerInfer-2: Fast Large Language Model Inference on a Smartphone*. 2024. arXiv: 2406.06282 [cs.LG].

[Yin+25] W. Yin, R. Yi, D. Xu, G. Huang, M. Xu, and X. Liu. "Elastic On-Device LLM Service." In: *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*. ACM MOBICOM '25. Kerry Hotel, Hong Kong, Hong Kong, China: Association for Computing Machinery, 2025, pp. 984–999. ISBN: 9798400711299. DOI: 10.1145/3680207.3765259.

[Yoo+23] W. Yoon, J. Ok, J. Oh, S. Moon, and Y. Kwon. "DiLOS: Do Not Trade Compatibility for Performance in Memory Disaggregation." In: *Proceedings of the Eighteenth European Conference on Computer Systems*. EuroSys '23. Rome, Italy: Association for Computing Machinery, 2023, pp. 266–282. ISBN: 9781450394871. DOI: 10.1145/3552326.3567488.

[Yua+24] J. Yuan, C. Yang, D. Cai, S. Wang, X. Yuan, Z. Zhang, X. Li, D. Zhang, H. Mei, X. Jia, S. Wang, and M. Xu. "Mobile Foundation Model as Firmware." In: *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. ACM MobiCom '24. Washington D.C., DC, USA: Association for Computing Machinery, 2024, pp. 279–295. ISBN: 9798400704895. DOI: 10.1145/3636534.3649361.